

Progetto di Laboratorio 2 A

Framework per analisi di file di testo adottando il paradigma MapReduce

a.a. 2025-26

Premessa

Il progetto descritto in questo documento è valevole per il superamento della prova di Laboratorio 2 A. Il testo deve essere letto e compreso nella sua interezza. Il progetto consiste nella realizzazione di un piccolo *framework*, scritto in linguaggio C, per l'analisi di file testuali su un singolo calcolatore. Il framework deve poter essere utilizzato, come una qualunque libreria, da un normale programma C. Il programma che usa il framework dovrà fornire due funzioni applicative: una funzione *mapper*, che elabora una riga di un file e produce informazioni associate ai token contenuti in quella riga, e una funzione *reducer*, che combina tutte le informazioni associate allo stesso token e produce il risultato finale relativo a quel token. Il framework dovrà occuparsi dell'esecuzione concorrente, della comunicazione tramite pipe, della creazione dei processi e della sincronizzazione dei thread; le funzioni applicative dovranno invece descrivere l'analisi specifica da svolgere sui dati. Non sono ammessi socket, memoria condivisa, `exec()` o thread POSIX `pthread`. Il progetto deve usare processi creati con `fork()`, pipe, `dup2()`, `waitpid()` e thread C11. È consentito l'uso dei semafori POSIX.

1 Introduzione concettuale al modello MapReduce

Il progetto si ispira a un modello di elaborazione chiamato *MapReduce*. Non è necessario conoscere preventivamente i dettagli di tale modello: in questa sezione ne viene data una descrizione concettuale, limitata agli aspetti necessari per comprendere l'assegnamento. MapReduce nasce per una classe molto comune di problemi: elaborare una grande quantità di dati, spesso testuali, suddividendo il lavoro in parti indipendenti, per poi combinare i risultati parziali in un risultato finale. L'idea non è legata a uno specifico linguaggio di programmazione o a una particolare tecnologia: è un modo di organizzare una computazione. Il principio fondamentale è separare il problema in due fasi:

1. una fase di *map*, nella quale i dati di input vengono trasformati in risultati intermedi;
2. una fase di *reduce*, nella quale i risultati intermedi appartenenti alla stessa categoria vengono combinati.

Fra queste due fasi esiste una terza operazione, spesso implicita ma essenziale: il raggruppamento dei risultati intermedi. Questa operazione raccoglie insieme tutti i dati che dovranno essere combinati dallo stesso reducer. Un modo utile per interpretare MapReduce è pensare ai risultati intermedi come coppie:

$\langle \text{chiave}, \text{valore} \rangle$

La chiave serve a stabilire quali valori devono essere combinati insieme. Il valore contiene invece l'informazione prodotta durante la fase di map. La funzione di map decide come produrre queste coppie; il sistema di esecuzione decide come raggrupparle; la funzione di reduce decide come combinare i valori di uno stesso gruppo. La fase di map può essere eseguita in parallelo

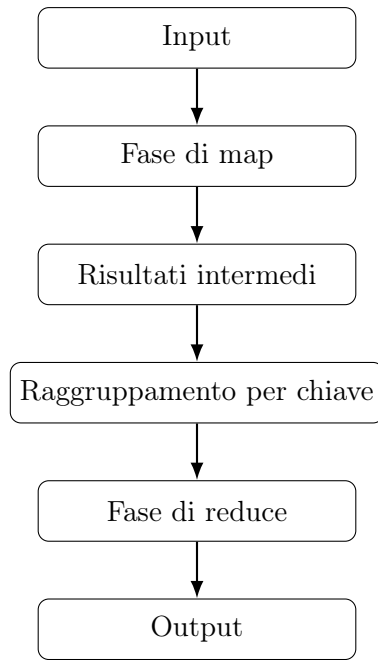


Figura 1: Schema concettuale del modello MapReduce.

perché ogni porzione di input può essere trasformata indipendentemente dalle altre. Due mapper diversi possono quindi lavorare su dati diversi senza doversi coordinare sul significato dell'analisi che stanno svolgendo.

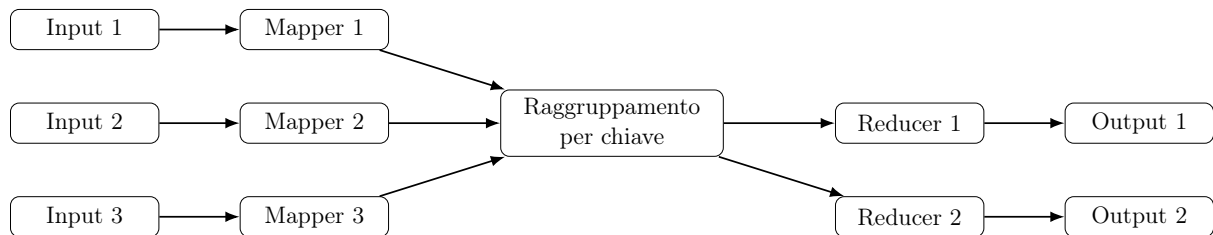


Figura 2: In un sistema MapReduce generale possono esserci più mapper e più reducer.

La fase di raggruppamento non consiste semplicemente nel mettere in ordine una lista di coppie. In un sistema con più mapper e più reducer, ogni mapper può produrre coppie associate a chiavi diverse; tali coppie devono essere inoltrate al reducer responsabile della rispettiva chiave. Il requisito fondamentale è che tutte le coppie con la stessa chiave siano elaborate dallo stesso reducer, anche se sono state prodotte da mapper diversi. La scelta del reducer a cui inviare una coppia viene tipicamente effettuata tramite una funzione di partizionamento, per esempio basata su una funzione hash applicata alla chiave. Il dettaglio della funzione non è importante in questa introduzione: ciò che conta è che la stessa chiave venga sempre assegnata allo stesso reducer.

1.1 Responsabilità

Dal punto di vista concettuale, MapReduce distingue tre responsabilità:

- la funzione *map* decide come trasformare l'input in risultati intermedi;
- il sistema di esecuzione raccoglie e raggruppa i risultati intermedi in base alla chiave;
- la funzione *reduce* decide come combinare i valori associati alla stessa chiave.

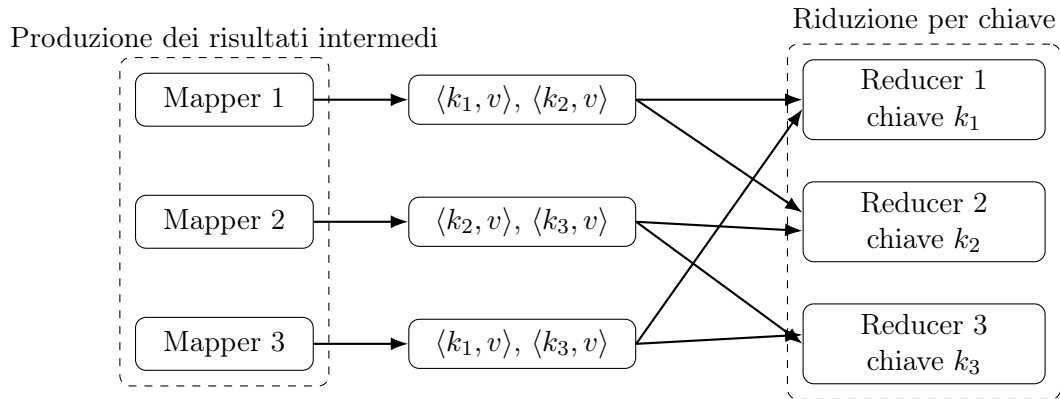


Figura 3: Distribuzione dei risultati intermedi: ogni mapper può produrre dati destinati a più reducer; tutte le occorrenze della stessa chiave devono arrivare allo stesso reducer.

Nel contesto di questo progetto non verrà realizzato un sistema distribuito: tutta l'esecuzione avviene sulla stessa macchina. Inoltre, per limitare la complessità, il progetto richiede un solo processo mapper e un solo processo reducer, entrambi multithread. Rimane però la stessa idea di base: separare la computazione in una fase di mapping, una fase di raggruppamento e una fase di reducing.

2 Descrizione generale del progetto

Si realizzi un framework denominato `libmr`, utilizzabile da programmi C esterni tramite un header pubblico `mr.h`. Il framework deve permettere di analizzare uno o più file di testo secondo il seguente modello:

`file_line` $\longrightarrow$ $\langle \text{token}, \text{processed_token} \rangle^* \longrightarrow \langle \text{token}, \text{processed_token}[] \rangle \longrightarrow \text{result}$

Il programma che usa il framework fornisce due funzioni:

- una funzione **mapper**, che riceve una riga logica di un file e può emettere zero o più coppie $\langle \text{token}, \text{processed_token} \rangle$;
- una funzione **reducer**, che riceve un token e tutti i valori associati a quel token, e produce uno o più risultati finali.

Il framework deve occuparsi di:

- leggere l'input di testo una riga alla volta;
- creare la pipeline di processi;
- trasferire ogni riga al processo mapper tramite pipe;
- trasferire le coppie prodotte dal mapper al processo reducer tramite pipe;
- raggruppare i valori per token nel processo reducer;
- invocare la funzione reducer sui gruppi completi;
- raccogliere i risultati finali e scriverli in output;

- gestire correttamente errori, chiusura e duplicazione dei descrittori, sincronizzazione e terminazione.

Il framework non deve contenere logica specifica per il conteggio delle parole o per altri esempi forniti dai docenti. In fase di valutazione potranno essere usate funzioni mapper e reducer diverse da quelle di esempio, purché conformi all'interfaccia pubblica.

3 Token, valori intermedi e risultati

Il framework opera su file testuali. Ogni riga letta dai file viene inviata al mapper. Il mapper può emettere coppie della forma:

`<token,processed_token>`

Il **token** è una sequenza non vuota di caratteri alfanumerici ASCII:

A-Z, a-z, 0-9

Il token viene usato dal framework come chiave di raggruppamento. Tutti i valori associati allo stesso token devono essere raccolti insieme e consegnati alla funzione reducer in un'unica invocazione. Nel contratto dell'interfaccia pubblica, il **token** deve essere rappresentato come una stringa C valida, terminata da `'\0'`, composta esclusivamente da caratteri alfanumerici ASCII. Il carattere terminatore non fa parte del token logico. Il framework può quindi usare funzioni sulle stringhe per confrontare e ordinare i token, ma tale possibilità vale solo per i token, non per i valori intermedi. Il valore **processed_token** è invece un dato opaco. Il framework non deve interpretarlo. In particolare, non deve assumere che sia un intero, una stringa C, un dato stampabile, un dato di dimensione fissa, o una sequenza terminata da `'\0'`. Il framework deve trattarlo sempre come una sequenza di byte di lunghezza nota. Lo stesso vale per i risultati prodotti dal reducer: il reducer produce sequenze di byte già serializzate, che il framework deve trasportare e scrivere nel file di output senza interpretarne il contenuto.

4 Input

L'input dell'elaborazione è costituito da uno o più file di testo. Il framework deve poter ricevere come input:

- un singolo file regolare;
- una directory contenente file regolari.

Nel caso in cui l'input sia un singolo file regolare, il framework deve elaborare soltanto quel file. Nel caso in cui l'input sia una directory, il framework deve elaborare tutti i file regolari contenuti direttamente in essa. Non è richiesta la scansione ricorsiva delle sottodirectory. Quando l'input è una directory, i file devono essere considerati in ordine lessicografico rispetto al loro nome. Questo vincolo serve a rendere deterministico l'ordine di lettura dell'input. Il framework legge ciascun file come una sequenza di righe logiche. Una riga logica è normalmente separata dalla successiva dal carattere `'\n'`, che non fa parte del contenuto passato al mapper. L'ultima riga di un file può anche non essere terminata da `'\n'`. La riga logica è l'unità di input passata alla funzione mapper. Il framework non deve suddividere preventivamente la riga in token: l'estrazione dei token e la produzione delle coppie

`<token,processed_token>`

sono responsabilità della funzione mapper fornita dal programma utente. Ogni riga inviata alla funzione mapper deve essere rappresentata tramite una struttura contenente almeno:

- il nome del file da cui la riga proviene;
- il numero della riga all'interno del file, partendo da 1;
- il contenuto della riga;
- la lunghezza in byte della riga.

La struttura dati passata dal framework alla funzione mapper utente, per rappresentare una riga logica è la seguente:

```

1  /* Riga logica di un file. */
2  typedef struct {
3      const char *file_name;
4      size_t file_name_len;
5      unsigned long line_number;
6      const char *line;
7      size_t line_len;
8  } mr_file_line_t;

```

La struttura `mr_file_line_t` rappresenta una riga logica nel processo mapper, nel momento in cui viene invocata la funzione mapper. Non deve essere usata come formato binario da scrivere direttamente sulla pipe, poiché i campi `file_name` e `line` sono puntatori validi solo nello spazio di indirizzamento del processo che li ha creati. Fra processo principale e processo mapper la riga deve quindi essere trasmessa in forma serializzata, usando lunghezze e byte effettivi. Il processo mapper ricostruisce poi localmente una struttura `mr_file_line_t` valida prima di invocare la funzione mapper. Il campo `file_name` identifica il file di provenienza. Il campo `line_number` indica il numero della riga nel file, iniziando da 1. Il campo `line` punta al contenuto della riga, mentre `line_len` ne indica la lunghezza in byte. Il contenuto della riga passato al mapper non deve necessariamente essere terminato da `'\0'`. Il mapper deve quindi usare il campo `line_len` per conoscere la lunghezza effettiva della riga. L'eventuale presenza di un terminatore nullo aggiunto dall'implementazione non fa parte del contratto e non deve essere assunta dal programma utente. Lo stesso vale per il campo `file_name`. Il carattere di fine riga non fa parte del contenuto logico della riga passato al mapper. Per esempio, una riga del file terminata da `'\n'` deve essere passata al mapper senza tale carattere finale. Devono essere gestiti correttamente almeno i seguenti casi:

- file vuoti;
- righe vuote;
- file contenenti una sola riga;
- ultima riga di un file non terminata da `'\n'`;

Il framework deve preservare, per ciascun file, l'ordine delle righe lette. Tuttavia, poiché l'elaborazione successiva avviene in modo concorrente, il programma utente non deve assumere che l'ordine con cui le righe vengono elaborate dai thread mapper coincida necessariamente con l'ordine di lettura. L'output finale deve comunque rispettare i requisiti di determinismo specificati in seguito.

5 Architettura dei processi

Il framework deve realizzare una pipeline locale composta da tre processi. Il processo principale è il processo del programma che usa il framework. Quando viene avviata un'elaborazione, il framework deve creare due processi figli:

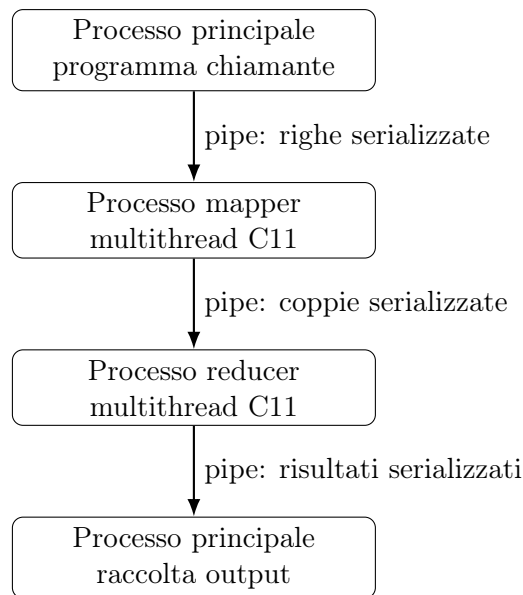


Figura 4: Pipeline richiesta dal progetto.

- un processo mapper, che riceve righe logiche dal processo principale, esegue la fase di mapping usando più thread C11 e produce coppie `<token, processed_token>`;
- un processo reducer, che riceve le coppie prodotte dal mapper, raggruppa i valori per token, esegue la fase di reducing usando più thread C11 e produce i risultati finali.

La comunicazione fra i tre processi deve avvenire tramite pipe. Sono quindi necessarie almeno tre pipe:

- una pipe dal processo principale al processo mapper;
- una pipe dal processo mapper al processo reducer;
- una pipe dal processo reducer al processo principale.

La pipe fra mapper e reducer viene chiusa dal processo mapper solo quando tutti i thread mapper hanno terminato. Tale chiusura costituisce il segnale, per il reducer, che non arriveranno più risultati intermedi e che il raggruppamento può essere completato. Un possibile schema di inizializzazione delle pipe è il seguente:

```

1  int main_to_mapper[2];
2  int mapper_to_reducer[2];
3  int reducer_to_main[2];
4
5  pipe(main_to_mapper);
6  pipe(mapper_to_reducer);
7  pipe(reducer_to_main);

```

Il codice riportato è solo indicativo, inoltre nell'implementazione reale ogni chiamata di sistema deve essere controllata e gli errori devono essere gestiti. Dopo la creazione delle pipe, il framework deve creare il processo mapper tramite `fork()`. Nel processo figlio, lo standard input deve essere collegato alla pipe proveniente dal processo principale, mentre lo standard output deve essere collegato alla pipe diretta al reducer.

```

1  pid_t mapper_pid = fork();
2

```

```

3  if (mapper_pid == 0) {
4      dup2(main_to_mapper[0], STDIN_FILENO);
5      dup2(mapper_to_reducer[1], STDOUT_FILENO);
6
7      /*
8       * Chiudere qui tutti i descrittori non necessari.
9       */
10
11     mapper_process_main(...);
12     _exit(0);
13 }

```

Analogamente, il framework deve creare il processo reducer. Nel processo reducer, lo standard input deve essere collegato alla pipe proveniente dal mapper, mentre lo standard output deve essere collegato alla pipe diretta al processo principale.

```

1  pid_t reducer_pid = fork();
2
3  if (reducer_pid == 0) {
4      dup2(mapper_to_reducer[0], STDIN_FILENO);
5      dup2(reducer_to_main[1], STDOUT_FILENO);
6
7      /*
8       * Chiudere qui tutti i descrittori non necessari.
9       */
10
11     reducer_process_main(...);
12     _exit(0);
13 }

```

Le funzioni `mapper_process_main()` e `reducer_process_main()` rappresentano il codice interno del framework eseguito nei processi figli. Non sono funzioni dell'interfaccia pubblica. In tali funzioni vengono creati i thread C11 necessari allo stadio mapper o allo stadio reducer. Nel processo principale devono rimanere aperti solo i descrittori effettivamente necessari:

- il lato di scrittura della pipe verso il mapper;
- il lato di lettura della pipe proveniente dal reducer.

Tutti gli altri descrittori devono essere chiusi. La corretta chiusura dei descrittori è parte essenziale del progetto: un descrittore lasciato aperto nel processo sbagliato può impedire la ricezione dell'EOF e causare un blocco permanente della pipeline. Il framework non deve usare `exec()`. Mapper e reducer sono funzioni C passate dal programma utente al framework. Con `fork()`, i processi figli ereditano il codice, la configurazione e i puntatori a funzione necessari per invocare tali callback. Con `exec()`, invece, l'immagine del processo verrebbe sostituita e questo modello non sarebbe più applicabile. Il framework non deve inoltre eseguire `fork()` da un processo che abbia già creato thread C11 attivi. I processi mapper e reducer devono essere creati prima dell'avvio dei rispettivi thread interni. Riassumendo, durante l'avvio dell'elaborazione il framework deve:

1. creare le pipe necessarie;
2. creare il processo mapper tramite `fork()`;
3. creare il processo reducer tramite `fork()`;
4. usare `dup2()` nei figli per collegare le pipe a `stdin` e `stdout`;

5. chiudere in ogni processo tutti i descrittori non utilizzati;
6. avviare nei processi figli i rispettivi thread C11;
7. inviare dal processo principale le righe serializzate al mapper;
8. raccogliere nel processo principale i risultati serializzati prodotti dal reducer;
9. attendere la terminazione dei processi figli tramite `waitpid()`.

5.1 Segnalazione della fine dell'input

La fine dell'input non deve essere segnalata tramite messaggi speciali, ma tramite la chiusura delle pipe e la conseguente ricezione di EOF.

Il flusso previsto è il seguente:

1. il processo principale legge tutti i file di input e invia al processo mapper le righe serializzate;
2. quando non ci sono più righe da inviare, il processo principale chiude il lato di scrittura della pipe verso il mapper;
3. il processo mapper riceve EOF sul proprio standard input;
4. il thread lettore del processo mapper marca la coda interna delle righe come chiusa;
5. i thread mapper completano l'elaborazione delle righe già presenti nella coda e terminano;
6. solo dopo la terminazione di tutti i thread mapper, il processo mapper chiude il proprio standard output, cioè la pipe verso il reducer;
7. il processo reducer riceve EOF sul proprio standard input;
8. a questo punto il reducer sa che non arriveranno più coppie `(token, processed_token)`;
9. il processo reducer completa il raggruppamento, invoca la funzione reducer sui gruppi completi e scrive i risultati finali;
10. infine il reducer chiude il proprio standard output, permettendo al processo principale di ricevere EOF sulla pipe dei risultati.

Nel processo mapper, i singoli thread mapper non devono chiudere la pipe verso il reducer. La chiusura deve avvenire una sola volta, da parte del codice di coordinamento del processo mapper, dopo che tutti i thread che possono produrre coppie sono terminati. In questo modo si evita che il reducer riceva EOF prima che tutte le coppie siano state effettivamente scritte.

Analogamente, il processo principale non deve mantenere aperto alcun descrittore di scrittura della pipe mapper-reducer. Se anche un solo processo mantiene aperto un lato di scrittura non utilizzato, il reducer potrebbe non ricevere mai EOF e rimanere bloccato in lettura.

6 Thread C11

Il processo mapper e il processo reducer devono essere multithread e devono usare i thread C11, tramite l'header:

```
1 #include <threads.h>
```


Non devono essere usati `pthread_create`, `pthread_mutex_t` o `pthread_cond_t`. Le funzioni passate a `thrd_create()` devono avere firma compatibile con `thrd_start_t`, cioè devono ricevere un unico argomento `void *` e restituire un `int`:

```
1 static int mapper_worker_main(void *arg);
2 static int reducer_worker_main(void *arg);
3 static int reader_main(void *arg);
```

Le funzioni mapper e reducer fornite dal programma utente non sono funzioni thread: sono funzioni di livello applicativo invocate dai thread worker. Le code usate all'interno dei processi mapper e reducer sono strutture dati interne al singolo processo e servono a coordinare i thread C11 secondo lo schema produttore-consumatore. Tali code non sono pipe e non sono memoria condivisa fra processi: sono normali strutture dati accessibili dai thread dello stesso processo e protette tramite `mtx_t` e `cnd_t`.

Il parametro `queue_size` indica la capacità massima, espressa in numero di elementi, delle code interne usate dal framework. Se una coda è piena, il thread produttore deve attendere; se una coda è vuota, il thread consumatore deve attendere. L'attesa deve essere realizzata tramite condition variable C11.

6.1 Processo mapper

Il processo mapper deve leggere dal proprio standard input messaggi contenenti righe logiche di file. Internamente deve usare una coda produttore-consumatore sincronizzata con `mtx_t` e `cnd_t`. Una possibile organizzazione è la seguente:

- un thread lettore legge le righe da `stdin` e le inserisce in una coda condivisa;
- un certo numero di thread mapper estrae righe dalla coda, invoca la funzione mapper e produce coppie `<token, processed_token>`;
- le coppie prodotte vengono scritte sullo standard output del processo mapper, cioè sulla pipe diretta al reducer; quando tutti i thread mapper sono terminati, il processo mapper chiude tale pipe per segnalare al reducer la fine dei risultati intermedi.

Poiché più thread mapper possono produrre coppie contemporaneamente, la scrittura verso la pipe deve essere sincronizzata. Un messaggio logico non deve essere mescolato con un messaggio prodotto da un altro thread.

6.2 Processo reducer

Il processo reducer deve leggere dal proprio standard input le coppie prodotte dal mapper. Anche il processo reducer deve essere multithread. Una possibile organizzazione è la seguente:

- un thread lettore legge le coppie da `stdin`;
- il reducer raggruppa tutti i valori associati allo stesso token;
- quando l'input è terminato, vengono creati gruppi della forma `<token, processed_token[]>`;
- i thread reducer elaborano i gruppi invocando la funzione reducer fornita dal programma utente;
- i risultati prodotti vengono raccolti e scritti verso lo standard output del processo reducer, cioè verso il processo principale.

La funzione reducer non deve essere invocata separatamente per ogni singola coppia prodotta dal mapper. Deve essere invocata solo dopo che il framework ha raccolto tutti i valori associati a un token.

7 Protocollo interno sulle pipe

La comunicazione tra processi deve usare un protocollo esplicito basato su lunghezze. Non è sufficiente scrivere testo libero sulle pipe, perché `processed_token` e `result` possono contenere byte arbitrari. Per esempio, una coppia prodotta dal mapper può essere rappresentata da un header:

```
1 typedef struct {  
2     int token_len;  
3     int value_len;  
4 } mr_pair_header_t;
```

seguito da:

- `token_len` byte contenenti il token;
- `value_len` byte contenenti il valore opaco.

La lunghezza `token_len` non include il terminatore `'\0'`. Il processo che riceve il token deve quindi allocare, o predisporre, un buffer di almeno `token_len + 1` byte e aggiungere localmente il terminatore nullo prima di usare il token come stringa C. Il valore `value_len`, invece, indica esattamente il numero di byte del valore opaco e non implica alcun terminatore. Il formato esatto dei messaggi deve essere documentato nella relazione. Il protocollo deve gestire correttamente letture e scritture parziali. Non è corretto assumere che una singola `read()` o una singola `write()` trasferisca un intero messaggio logico. Devono quindi essere realizzate funzioni ausiliarie equivalenti a:

```
1 ssize_t readn(int fd, void *buf, size_t n);  
2 ssize_t writen(int fd, const void *buf, size_t n);
```

8 Output

Il file di output deve essere prodotto dal framework. Per preservare la generalità del modello, i risultati emessi dal reducer devono essere scritti in un formato a record con lunghezze esplicite. Ogni record di output deve contenere almeno:

- la lunghezza del token;
- il token;
- la lunghezza del risultato;
- i byte del risultato.

Il framework deve garantire un output deterministico. A parità di input, funzioni mapper/-reducer e parametri del framework, il file di output deve essere identico tra esecuzioni diverse. Una scelta semplice è scrivere i risultati ordinati lessicograficamente per token. Se il reducer emette più risultati per lo stesso token, l'ordine relativo deve essere definito e documentato. Gli esempi forniti potranno includere un programma ausiliario per stampare in forma leggibile il contenuto del file di output, ma la correttezza del framework non deve dipendere dal fatto che i risultati siano stringhe C. Anche nel formato di output, la lunghezza del token non include il terminatore `'\0'`.

9 Analisi di esempio

Deve essere fornito almeno un programma di esempio che usi il framework. L'esempio consigliato è il conteggio delle occorrenze dei token.

In questo esempio, il mapper emette per ogni token il valore intero 1, serializzato come byte:

```
1 int one = 1;
2 emit(token, &one, sizeof(one), emit_arg);
```

Il reducer riceve tutti i valori associati allo stesso token, li interpreta come interi e produce il totale.

Questo esempio serve soltanto a illustrare l'uso del framework. Il framework non deve contenere logica specifica per il conteggio delle parole. In fase di valutazione potranno essere usate funzioni mapper e reducer diverse, non note in anticipo, purché conformi all'interfaccia pubblica.

10 Precisazioni importanti sul contratto del framework

Le seguenti precisazioni fanno parte integrante della specifica:

- il framework deve trattare ogni `processed_token` come una sequenza opaca di byte di lunghezza nota;
- non è garantito che i valori intermedi siano stringhe C terminate da `'\0'`;
- i valori intermedi possono contenere byte nulli;
- il framework non deve usare funzioni come `strlen`, `strcpy`, `strcmp` o `printf("%s")` sui valori intermedi;
- il token è l'unico campo interpretato dal framework, ed è usato esclusivamente per il raggruppamento;
- la funzione reducer deve essere invocata una sola volta per ogni token distinto, ricevendo tutti i valori associati a quel token;
- non è corretto invocare il reducer separatamente per ogni singola coppia prodotta dal mapper;
- il mapper e il reducer forniti come esempio non definiscono il caso generale;
- la comunicazione su pipe deve gestire correttamente letture e scritture parziali;
- l'uso di thread e processi non autorizza race condition, perdita di risultati, duplicazioni o dipendenza dall'ordine di scheduling.
- Nei processi mapper e reducer lo standard output è riservato al protocollo interno del framework. Le funzioni applicative mapper e reducer non devono scrivere direttamente su `stdout`; eventuali messaggi diagnostici devono usare il log del framework o `stderr`.
- Le lunghezze contenute negli header del protocollo sono espresse come valori di tipo `int`. Ogni lunghezza ricevuta deve essere controllata prima di essere usata: non sono ammessi valori negativi, e l'implementazione deve rifiutare messaggi la cui lunghezza ecceda i limiti ragionevoli scelti e documentati. Prima di convertire tali valori a `size_t`, il framework deve verificarne la validità.

11 Log di esecuzione

Il framework deve produrre un file di log. Il nome del file di log può essere configurato tramite `mr_attr_set_log_file()`. Se non specificato, il framework deve usare un nome di default ragionevole, ad esempio `mr.log`.

Ogni riga del log deve avere un formato chiaro e documentato, ad esempio:

```
[timestamp] [processo] [thread] [evento] messaggio
```

Devono essere registrati almeno:

- creazione delle pipe;
- creazione dei processi mapper e reducer;
- avvio e terminazione dei thread;
- apertura e chiusura dei file di input e output;
- numero di righe inviate al mapper;
- numero di coppie prodotte dal mapper;
- numero di token distinti raggruppati dal reducer;
- numero di risultati finali prodotti;
- errori rilevati.

Se più thread o più processi scrivono sul log, l'accesso deve essere sincronizzato. Nel caso di scritture da processi diversi, devono essere usati meccanismi adatti alla sincronizzazione inter-processo, ad esempio semafori POSIX, lock su file, oppure una soluzione in cui gli eventi di log vengano raccolti e scritti da un unico processo.

12 Requisiti tecnici

Il progetto deve essere realizzato in linguaggio C e deve compilare ed eseguire correttamente su Linux Ubuntu 24.04, in particolare sulla macchina di laboratorio indicata dai docenti. Ogni chiamata di sistema deve essere opportunamente controllata. Ove appropriato, si consiglia l'uso di macro o funzioni wrapper per uniformare la gestione degli errori.

13 Test e valutazione

Il progetto deve produrre una libreria statica `libmr.a`. La valutazione potrà compilare programmi di test esterni che includono `mr.h`, collegano `libmr.a` e forniscono mapper e reducer propri. Il framework sarà valutato anche con mapper e reducer diversi da quelli distribuiti come esempio. Non saranno considerate corrette implementazioni che funzionano soltanto con l'esempio di conteggio delle parole. Il target `make test` deve eseguire una batteria di test automatizzati predisposti dallo studente.

14 Addendum per chi non ha superato un numero congruo di prove in itinere

Agli studenti che non abbiano superato un numero congruo di prove in itinere sarà richiesto di implementare alcune funzionalità aggiuntive. Tali funzionalità sono le seguenti:

- supporto alla scansione ricorsiva delle directory di input;
- possibilità di eseguire più elaborazioni indipendenti nello stesso processo chiamante, senza interferenze fra le diverse istanze di `mr_t`;
- statistiche finali in un file separato, contenente almeno tempi di esecuzione, numero di righe lette, numero di coppie prodotte, numero di token distinti e numero di risultati emessi;
- possibilità per l'utente di specificare una funzione di hashing deterministica per l'assegnazione dei token ai thread reducer interni al processo reducer. Se non viene specificata alcuna funzione di hashing, il framework deve usare una funzione di default. La funzione di hashing deve essere deterministica: due occorrenze dello stesso token devono essere assegnate allo stesso thread reducer. La funzione non deve modificare il token né dipendere dall'ordine di esecuzione dei thread.

Poiché `mr_attr_t` è un tipo pubblico, gli studenti che implementano l'addendum dovranno estenderlo aggiungendo anche i campi necessari a memorizzare la funzione di hashing e il relativo argomento utente. Nel caso dell'addendum, la definizione di `mr_attr_t` riportata nell'interfaccia pubblica di base deve essere estesa come segue:

```
1 typedef size_t (*mr_hash_t)(
2     const char *token,
3     size_t token_len,
4     void *user_arg
5 );
6
7 typedef struct {
8     size_t mapper_threads;
9     size_t reducer_threads;
10    size_t queue_size;
11    const char *log_file;
12
13    mr_hash_t hash;
14    void *hash_arg;
15 } mr_attr_t;
16
17 int mr_attr_set_hash_function(
18     mr_attr_t *attr,
19     mr_hash_t hash,
20     void *hash_arg
21 );
```

Anche queste funzionalità devono essere documentate nella relazione e coperte da test.

15 Consegna

Il progetto deve essere consegnato come archivio `.zip`. L'archivio deve contenere almeno:

- file sorgenti `.c` e `.h`;

- directory `include/` contenente `mr.h`;
- directory `src/` contenente l'implementazione del framework;
- directory `examples/` contenente almeno un esempio d'uso;
- `Makefile` e `README`;
- relazione in formato PDF.

Il `Makefile` deve prevedere almeno i seguenti target:

- `make`: compila il framework, la libreria `libmr.a` e almeno un esempio;
- `make test`: esegue i test automatici;
- `make clean`: rimuove file oggetto, eseguibili e file temporanei.

Scompattando l'archivio deve essere possibile compilare il codice tramite `make` senza interventi manuali.

16 Relazione

La documentazione del progetto consiste nei commenti al codice e in una breve relazione, in formato PDF, di massimo 10 pagine. La relazione non deve ripetere integralmente le specifiche contenute in questo documento. La relazione deve contenere:

- descrizione dell'architettura generale del framework;
- descrizione dell'interfaccia pubblica;
- organizzazione dei processi e uso di `fork()`, `pipe()`, `dup2()` e `waitpid()`;
- organizzazione dei thread C11 nel mapper e nel reducer;
- struttura delle code interne e meccanismi di sincronizzazione;
- formato dei messaggi scambiati sulle pipe;
- struttura dati usata per il raggruppamento per token;
- formato del file di output;
- formato del file di log;
- descrizione dei test realizzati;

17 Note finali

Il progetto deve essere svolto in autonomia. Il codice consegnato deve essere comprensibile, commentato in modo adeguato e organizzato in più file secondo criteri ragionevoli. L'uso di strumenti automatici di supporto alla programmazione non solleva lo studente dalla responsabilità di comprendere, verificare e saper spiegare il codice consegnato. La valutazione potrà includere test con mapper e reducer non noti in anticipo e domande sulle scelte progettuali adottate.

A Interfaccia pubblica richiesta

Il framework deve esporre un header pubblico `include/mr.h`. Di seguito è riportata l'interfaccia minima richiesta. Sono ammesse funzioni ausiliarie aggiuntive, purché non modifichino la semantica di quelle qui specificate.

```
1  #ifndef MR_H
2  #define MR_H
3
4  #include <stddef.h>
5
6  /* Handle opaco di una elaborazione. */
7  typedef struct mr *mr_t;
8
9  /*
10 * Attributi di configurazione del framework.
11 *
12 * mapper_threads:
13 *     numero di thread usati nel processo mapper.
14 *
15 * reducer_threads:
16 *     numero di thread usati nel processo reducer.
17 *
18 * queue_size:
19 *     dimensione delle code interne usate dal framework.
20 *
21 * log_file:
22 *     nome del file di log, oppure NULL per usare il nome di default.
23 */
24
25 typedef struct {
26     size_t mapper_threads;
27     size_t reducer_threads;
28
29     /*
30      * Capacita' massima delle code interne usate per coordinare
31      * i thread C11 nei processi mapper e reducer.
32      * Non rappresenta la dimensione delle pipe.
33      */
34     size_t queue_size;
35
36     const char *log_file;
37 } mr_attr_t;
38
39 /*
40 * Riga logica di un file, vista dalla funzione mapper.
41 *
42 * I campi file_name e line sono puntatori validi nel processo mapper
43 * solo durante l'invocazione della funzione mapper.
44 *
45 * file_name e line non devono necessariamente essere terminati da
46 * '\0'.
47 * Le rispettive lunghezze sono indicate da file_name_len e line_len.
48 */
49 typedef struct {
50     const char *file_name;
51     size_t file_name_len;
52     unsigned long line_number;
```

```

52     const char *line;
53     size_t line_len;
54 } mr_file_line_t;
55
56 /*
57  * Valore opaco associato a un token.
58  *
59  * Il framework non interpreta il contenuto di data.
60  * Se size vale 0, data puo' essere NULL.
61  */
62 typedef struct {
63     const void *data;
64     size_t size;
65 } mr_value_t;
66
67 /*
68  * Funzione usata dal mapper per emettere una coppia <token, valore>.
69  *
70  * token deve essere una stringa C valida, terminata da '\0',
71  * composta soltanto da caratteri alfanumerici ASCII.
72  *
73  * value e' una sequenza opaca di byte di lunghezza value_size.
74  * Se value_size vale 0, value puo' essere NULL.
75  */
76 typedef int (*mr_emit_pair_t)(
77     const char *token,
78     const void *value,
79     size_t value_size,
80     void *emit_arg
81 );
82
83 /*
84  * Funzione usata dal reducer per emettere un risultato finale.
85  *
86  * token deve essere una stringa C valida. Nel progetto base, il token
87  * emesso dal reducer deve coincidere con il token ricevuto dalla
88  * funzione
89  * reducer.
90  *
91  * result e' una sequenza opaca di byte di lunghezza result_size.
92  * Se result_size vale 0, result puo' essere NULL.
93  */
94 typedef int (*mr_emit_result_t)(
95     const char *token,
96     const void *result,
97     size_t result_size,
98     void *emit_arg
99 );
100
101 /*
102  * Funzione mapper fornita dal programma utente.
103  *
104  * La funzione mapper riceve una riga logica e puo' emettere zero o piu
105  * ,
106  * coppie <token, valore> usando la funzione emit.
107  */
108 typedef int (*mr_mapper_t)(
109     const mr_file_line_t *line,

```



```

108     mr_emit_pair_t emit,
109     void *emit_arg,
110     void *user_arg
111 );
112
113 /*
114  * Funzione reducer fornita dal programma utente.
115  *
116  * La funzione reducer riceve un token e tutti i valori associati a
117  * quel
118  * token. Puo' emettere zero o piu' risultati usando la funzione emit.
119  */
120 typedef int (*mr_reducer_t)(
121     const char *token,
122     const mr_value_t *values,
123     size_t values_count,
124     mr_emit_result_t emit,
125     void *emit_arg,
126     void *user_arg
127 );
128
129 int mr_attr_init(mr_attr_t *attr);
130 int mr_attr_destroy(mr_attr_t *attr);
131
132 int mr_attr_set_mapper_threads(mr_attr_t *attr, size_t n);
133 int mr_attr_set_reducer_threads(mr_attr_t *attr, size_t n);
134 int mr_attr_set_queue_size(mr_attr_t *attr, size_t n);
135 int mr_attr_set_log_file(mr_attr_t *attr, const char *path);
136
137 int mr_create(
138     mr_t *mr,
139     const mr_attr_t *attr,
140     mr_mapper_t mapper,
141     mr_reducer_t reducer,
142     void *user_arg
143 );
144
145 int mr_start(mr_t mr, const char *input_path, const char *output_path);
146 int mr_destroy(mr_t mr);
147 #endif

```

Le funzioni del framework devono restituire 0 in caso di successo e -1 in caso di errore, impostando `errno` ove appropriato. Il tipo `mr_t` è opaco: il programma applicativo non deve conoscere i campi interni della struttura `struct mr`. Il tipo `mr_attr_t`, invece, è completamente definito nell'header, così da poter essere usato come variabile locale dal programma utente. La funzione `mr_attr_init()` deve inizializzare gli attributi con valori di default validi. In particolare, il numero di thread mapper e reducer deve essere almeno 1. Le funzioni `mr_attr_set_*` devono rifiutare valori non validi, ad esempio un numero di thread pari a 0. La funzione `mr_attr_set_queue_size()` configura la dimensione massima delle code interne al framework. Tale parametro non modifica la dimensione delle pipe del sistema operativo. Un valore pari a 0 deve essere considerato non valido. La funzione `mr_create()` crea una nuova elaborazione del framework. Essa deve copiare internamente le informazioni di configurazione necessarie. Dopo il ritorno con successo di `mr_create()`, il chiamante può distruggere o modificare la struttura `mr_attr_t` senza alterare l'elaborazione già creata. La funzione `mr_start()` avvia l'elaborazione e restituisce il controllo al chiamante solo quando l'elaborazione è terminata.

oppure quando si verifica un errore. Durante tale esecuzione, il framework crea i processi, trasferisce i dati sulla pipeline, attende la terminazione dei processi figli e produce il file di output. La funzione `mr_destroy()` rilascia le risorse associate all'elaborazione.

A.1 Validità dei puntatori e copia dei dati

I puntatori passati dal framework alle funzioni mapper e reducer sono validi soltanto durante l'invocazione della callback. Se una funzione applicativa vuole conservare tali dati oltre il ritorno dalla callback, deve copiarli. Analogamente, quando una funzione mapper invoca `emit` per produrre una coppia

`<token, processed_token>`,

il framework deve copiare i dati ricevuti prima che `emit` ritorni. Il programma utente può quindi riutilizzare o liberare i buffer passati a `emit` subito dopo il ritorno della chiamata. Lo stesso vale per i risultati prodotti dal reducer tramite `mr_emit_result_t`: il framework deve copiare il risultato prima che la funzione di emissione ritorni. Il campo `user_arg` viene passato senza interpretazione alle funzioni mapper e reducer. Poiché il framework usa `fork()`, eventuali modifiche effettuate a oggetti puntati da `user_arg` nei processi figli non sono visibili nel processo principale né nell'altro processo figlio.

B Uso previsto del framework

Un programma applicativo che usa il framework dovrà avere una struttura simile alla seguente.

```
1 #include <stdio.h>
2 #include "mr.h"
3
4 int main(int argc, char **argv) {
5     mr_t mr;
6     mr_attr_t attr;
7
8     if (mr_attr_init(&attr) == -1) {
9         perror("mr_attr_init");
10        return 1;
11    }
12
13    if (mr_attr_set_mapper_threads(&attr, 4) == -1) {
14        perror("mr_attr_set_mapper_threads");
15        mr_attr_destroy(&attr);
16        return 1;
17    }
18
19    if (mr_attr_set_reducer_threads(&attr, 4) == -1) {
20        perror("mr_attr_set_reducer_threads");
21        mr_attr_destroy(&attr);
22        return 1;
23    }
24
25    if (mr_attr_set_queue_size(&attr, 64) == -1) {
26        perror("mr_attr_set_queue_size");
27        mr_attr_destroy(&attr);
28        return 1;
29    }
30
31    if (mr_attr_set_log_file(&attr, "mr.log") == -1) {
32        perror("mr_attr_set_log_file");
```

```

33     mr_attr_destroy(&attr);
34     return 1;
35 }
36
37 if (mr_create(&mr, &attr, my_mapper, my_reducer, NULL) == -1) {
38     perror("mr_create");
39     mr_attr_destroy(&attr);
40     return 1;
41 }
42
43 if (mr_start(mr, "input", "output.mro") == -1) {
44     perror("mr_start");
45     mr_destroy(mr);
46     mr_attr_destroy(&attr);
47     return 1;
48 }
49
50 mr_destroy(mr);
51 mr_attr_destroy(&attr);
52
53 return 0;
54 }

```

La funzione `mr_start()` è bloccante: quando restituisce 0, l'elaborazione è terminata correttamente e il file di output è stato prodotto. Se restituisce -1, si è verificato un errore e `errno` deve essere impostato ove appropriato.

B.1 Estensione dell'interfaccia per l'addendum

Per gli studenti tenuti a realizzare l'addendum, l'interfaccia pubblica dovrà essere estesa per consentire all'utente di specificare una funzione di hashing deterministica per assegnare i token ai thread reducer interni al processo reducer.

Una possibile estensione dell'header `mr.h` è la seguente:

```

1  /*
2   * Funzione di hashing opzionale per partizionare i token fra
3   * i thread reducer interni al processo reducer.
4   *
5   * La funzione deve essere deterministica: due occorrenze dello stesso
6   * token devono produrre lo stesso valore di hash.
7   */
8  typedef size_t (*mr_hash_t)(
9      const char *token,
10     size_t token_len,
11     void *user_arg
12 );
13
14 int mr_attr_set_hash_function(
15     mr_attr_t *attr,
16     mr_hash_t hash,
17     void *hash_arg
18 );

```

Se non viene specificata alcuna funzione di hashing, il framework deve usare una funzione di default. La funzione di hashing non deve modificare il token e non deve dipendere dall'ordine di esecuzione dei thread.